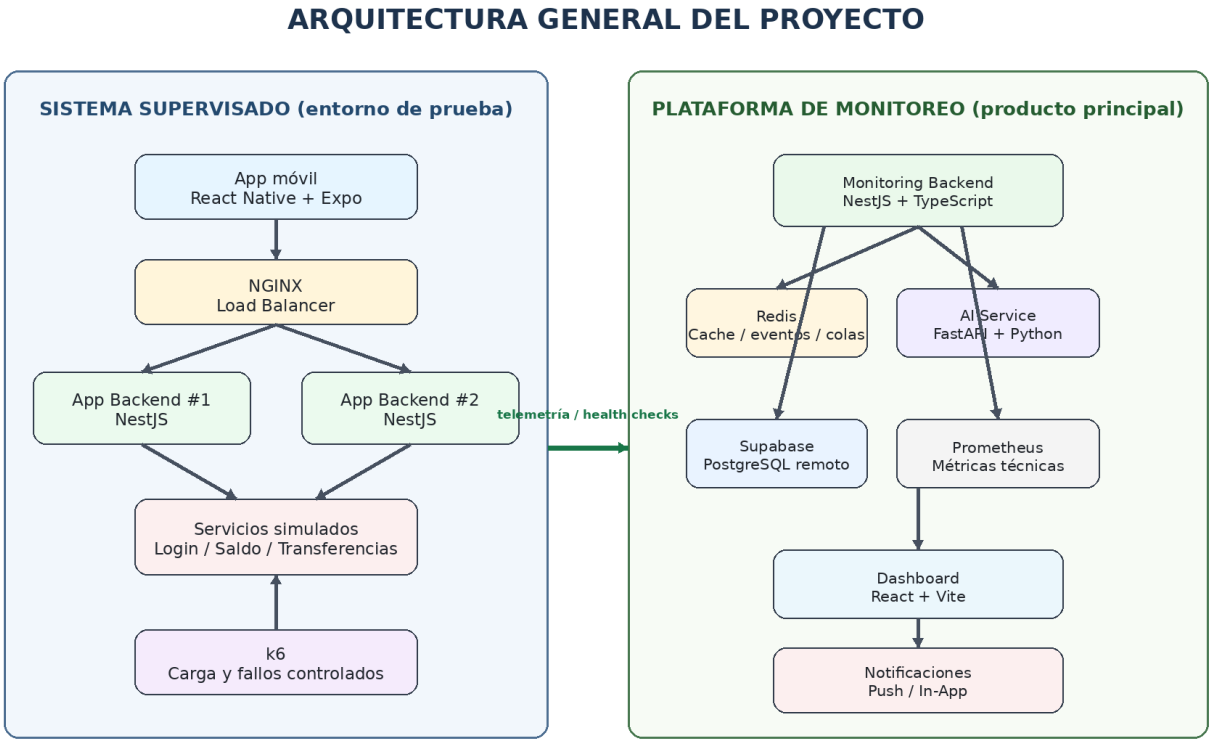
INFORME TÉCNICO DE ARQUITECTURA

Plataforma de Monitoreo y Gestión Proactiva de Interrupciones Digitales

Guía de arquitectura, componentes, repositorios, Docker, IA, datos, simulación y flujo operativo



Versión de trabajo para desarrollo universitario con enfoque profesional

1. Resumen ejecutivo

La solución no es únicamente una aplicación móvil ni únicamente un dashboard. El producto principal es una **plataforma de monitoreo y gestión proactiva de interrupciones** que observa servicios digitales simulados, detecta degradaciones y caídas, registra incidentes, comunica afectaciones y acompaña la recuperación del servicio.

Para demostrarla se construye un entorno supervisado controlado: una app móvil prototipo, un backend simulado con varias instancias detrás de NGINX y escenarios de carga/fallo. Ese entorno puede fallar; la plataforma de monitoreo debe continuar funcionando para observarlo.

Elemento	Qué representa	Idea simple
Sistema supervisado	App + servicios simulados + NGINX	Lo que podemos estresar o hacer fallar.
Plataforma de monitoreo	Monitoring Backend + IA + incidentes + alertas	El producto principal del proyecto.
Dashboard	Interfaz web para operadores	Muestra lo que detecta y gestiona la plataforma.
Supabase	PostgreSQL remoto	Persistencia compartida en Internet.
Docker	Entorno de ejecución	Empaqueta y levanta los componentes de forma reproducible.

2. ¿Qué es exactamente el proyecto?

Definición recomendada:

“Plataforma inteligente de monitoreo y gestión proactiva de interrupciones de servicios digitales, capaz de supervisar continuamente la disponibilidad y rendimiento de servicios, detectar degradaciones mediante reglas e inteligencia artificial, gestionar incidentes, comunicar afectaciones a usuarios y operadores, y realizar seguimiento hasta el restablecimiento.”

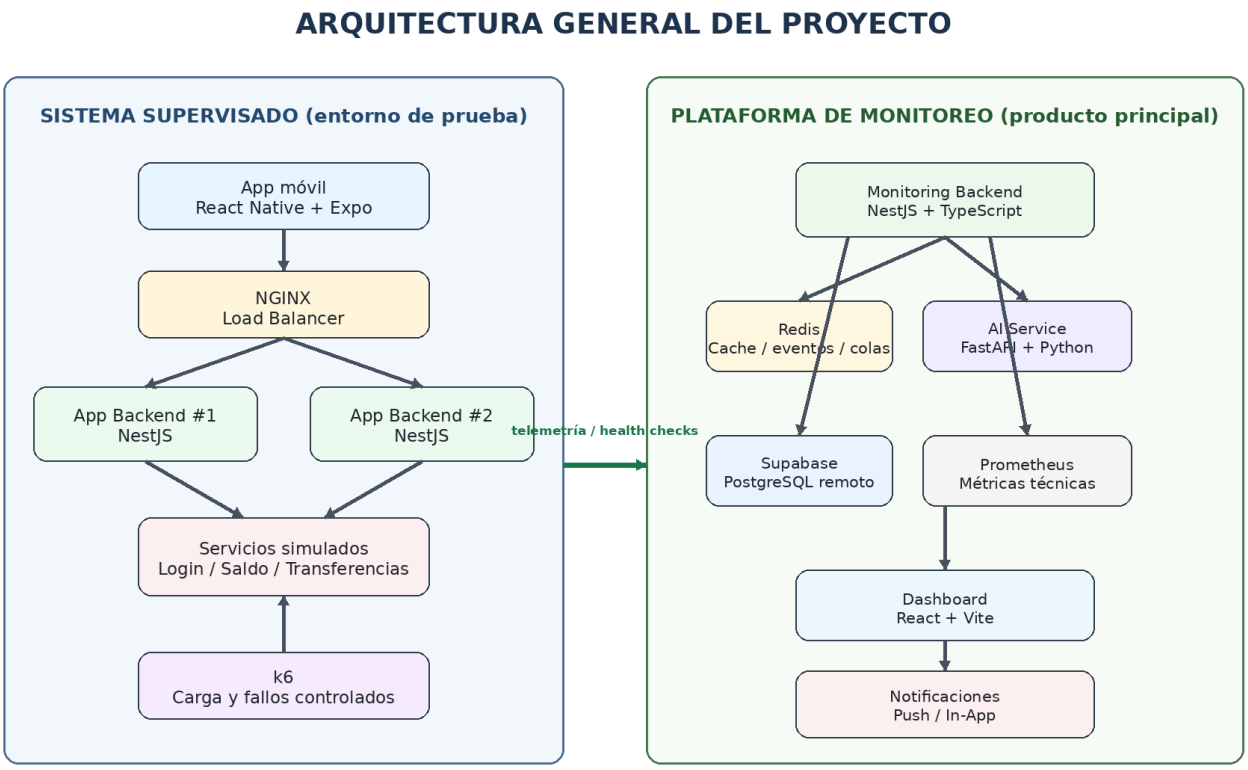
2.1 Lo que sí forma parte del valor principal

- Monitoreo continuo de servicios y health checks.
- Recolección y análisis de métricas.
- Detección de degradación e indisponibilidad.
- Apoyo de IA para detectar anomalías y nivel de riesgo.
- Creación y seguimiento de incidentes.
- Alertas y notificaciones a usuarios/operadores.
- Dashboard operativo e historial.
- Pruebas controladas de carga, degradación, caída y recuperación.

2.2 Lo que es solamente un entorno de prueba

La app móvil y el backend bancario simulado no buscan reconstruir un core bancario real. Se implementan solo las funciones necesarias para demostrar el comportamiento de la plataforma: por ejemplo login simulado, saldo simulado, transferencias simuladas y endpoints de salud.

3. Arquitectura general



3.1 Sistema A - Sistema supervisado

Es el sistema que será sometido a carga y fallos. Contiene la aplicación móvil, NGINX y dos instancias del mismo backend simulado. La finalidad es producir una situación observable y verificable.

3.2 Sistema B - Plataforma de monitoreo

Es el núcleo del proyecto. Contiene el Monitoring Backend, el motor de detección, reglas, integración con IA, incidentes, alertas, notificaciones, persistencia, dashboard y observabilidad. Debe permanecer activo aunque el sistema supervisado falle.

4. ¿Cuántos backends hay y por qué?

Existen dos tipos de backend, con responsabilidades diferentes:

Backend	Tecnología	Responsabilidad	¿Se hace fallar?
App Backend simulado	NestJS + TypeScript	Login/saldo/	Sí, intencionalmente.

		transferencias simuladas, health endpoints y comportamiento de prueba.	
Monitoring Backend	NestJS + TypeScript	Monitoreo, estado, incidentes, reglas, alertas, notificaciones, usuarios operadores y API del dashboard.	No durante la prueba principal.

Importante: App Backend #1 y App Backend #2 no son dos proyectos distintos. Son dos contenedores creados desde la misma imagen Docker y el mismo repositorio.

5. ¿Dónde está la Inteligencia Artificial?

La IA pertenece conceptualmente a la plataforma de monitoreo, pero se implementa como un servicio separado para aprovechar el ecosistema de Python. El Monitoring Backend la consulta cuando dispone de métricas suficientes.

Componente	Tecnología	Responsabilidad
Monitoring Backend	NestJS / TypeScript	Coordina reglas, estado, incidentes y decisión final.
AI Service	Python + FastAPI	Expone API interna de análisis.
Modelo inicial	Scikit-learn / Isolation Forest	Detecta comportamiento anómalo y devuelve score/riesgo.

La IA no controla NGINX y no debe ser la única responsable de declarar una caída. Su función es aportar evidencia: “este patrón es anómalo y su riesgo es alto”. La decisión operacional combina reglas + métricas + resultado de IA.

6. Arquitectura de datos: Supabase, Redis y Prometheus

Para el trabajo colaborativo se propone usar Supabase únicamente como PostgreSQL administrado en Internet. Los backends siguen siendo aplicaciones NestJS ejecutadas localmente o en una PC de demostración.

Tecnología	Qué guarda	Qué NO debe hacer
Supabase / PostgreSQL	Usuarios, configuración, servicios, incidentes, historial, auditoría, notificaciones y evidencias persistentes.	No almacenar cada request individual de k6.
Redis	Estado reciente, caché, colas, eventos, deduplicación y trabajos asíncronos.	No reemplazar la persistencia histórica.
Prometheus	Series temporales y métricas técnicas: requests, latencia,	No actuar como base de negocio.

	CPU, memoria, contenedores.	

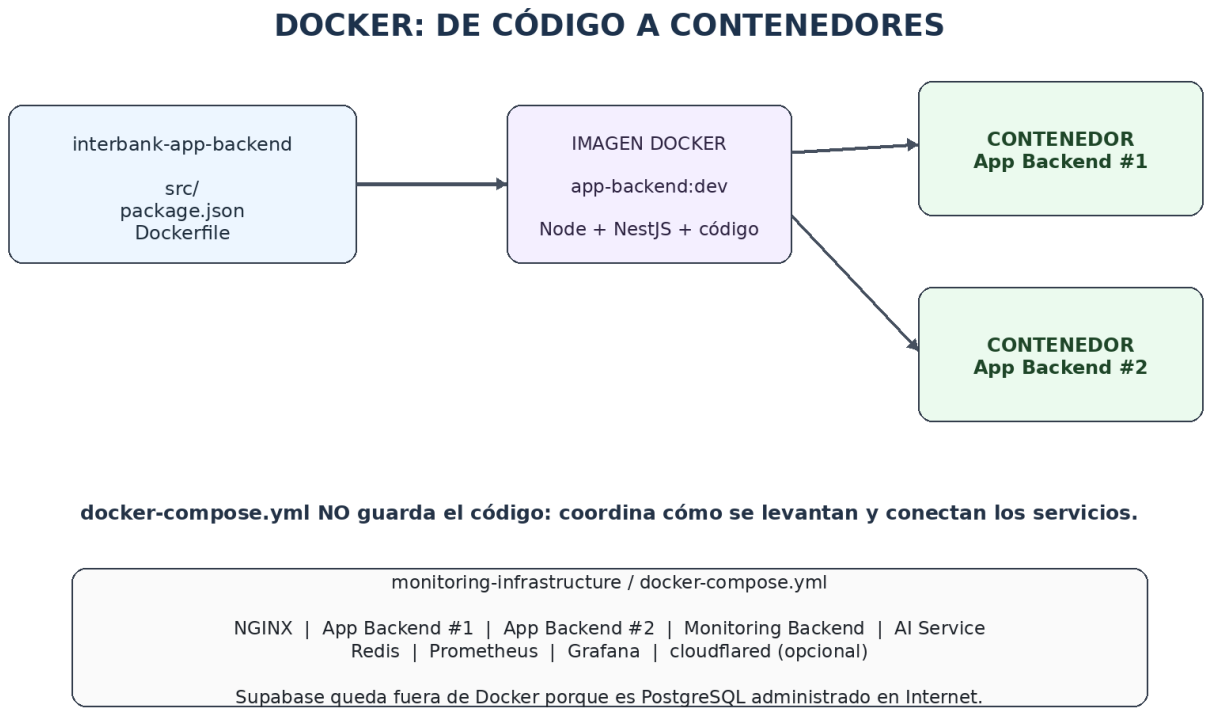
6.1 ¿La base se saturará durante las simulaciones?

No debería, si se evita guardar cada petición de k6. El generador puede producir miles de solicitudes, pero el sistema de monitoreo debe trabajar con métricas agregadas por ventanas (por ejemplo 10 s): cantidad de solicitudes, error rate, p95, timeouts y disponibilidad. PostgreSQL conserva eventos relevantes e historial; Redis y Prometheus manejan información de alta frecuencia.

6.2 Organización recomendada en Supabase

Para un equipo universitario: un proyecto Supabase compartido de desarrollo y separación lógica por esquemas o prefijos (por ejemplo app_sim y monitoring). Si el tiempo lo permite, mantener además un proyecto Supabase “demo” estable. Las migraciones deben versionarse desde Prisma/Git y no depender de cambios manuales en el panel.

7. Docker explicado de forma simple



Regla sencilla: el Dockerfile explica cómo construir UNA aplicación; Docker Compose explica cómo ejecutar y conectar MUCHOS servicios.

- Dockerfile = receta para crear una imagen.
- Imagen = plantilla ejecutable con código, runtime y dependencias.
- Contenedor = una ejecución concreta de esa imagen.
- Una misma imagen del App Backend puede generar App Backend #1 y #2.
- docker-compose.yml = orquestación local del ecosistema.

7.1 ¿Qué se ejecuta en Docker?

Inicialmente: NGINX, App Backend #1, App Backend #2, Monitoring Backend, AI Service, Redis, Prometheus, Grafana y, si se desea, cloudflared. Supabase queda fuera porque es un servicio remoto. La app React Native + Expo puede ejecutarse directamente en el equipo/dispositivo durante desarrollo.

8. Repositorios profesionales propuestos

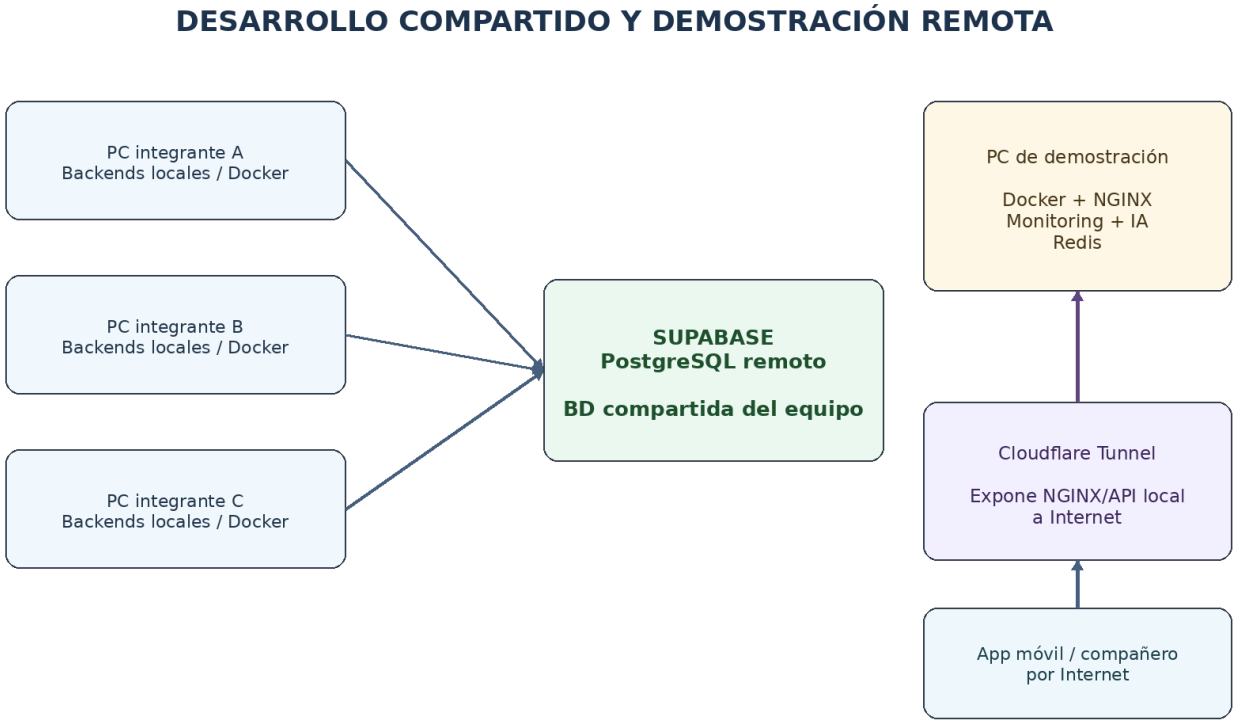
Repositorio	Stack principal	Responsabilidad
01 interbank-mobile	React Native + Expo	Experiencia del cliente y mensajes de afectación.
02 interbank-app-backend	NestJS	Servicios bancarios simulados y health endpoints; de aquí salen varias instancias.
03 monitoring-backend	NestJS	Corazón del proyecto: monitoreo, estado, incidentes, reglas, notificaciones y API del dashboard.
04 monitoring-ai	Python + FastAPI	Análisis de anomalías, entrenamiento y predicción.
05 monitoring-dashboard	React + Vite	Interfaz de operación: servicios, incidentes, métricas, IA e historial.
06 monitoring-infrastructure	Docker Compose + NGINX + Prometheus + Grafana + k6	Define cómo se levanta, conecta, observa y prueba el ecosistema.

8.1 ¿Qué contiene monitoring-infrastructure?

```
monitoring-infrastructure/
├── docker-compose.yml
├── docker-compose.dev.yml
├── .env.example
├── nginx/
│   └── nginx.conf
├── prometheus/
│   └── prometheus.yml
├── grafana/
│   ├── provisioning/
│   └── dashboards/
├── k6/
│   ├── smoke.js
│   ├── load.js
│   ├── stress.js
│   └── failover.js
└── scripts/
    ├── up.sh
```

```
├── down.sh
└── reset.sh
```

9. Trabajo colaborativo, Supabase y Cloudflare Tunnel



Supabase permite que todos los integrantes se conecten a la misma base PostgreSQL por Internet. Cada integrante puede ejecutar el backend localmente. Para una demostración remota, una PC puede dejar Docker/NGINX activos y usar Cloudflare Tunnel para exponer la API local temporalmente.

Cloudflare Tunnel no sustituye el hosting: si la PC de demostración se apaga, el backend local y el túnel dejan de estar disponibles. Supabase seguirá disponible porque es remoto.

10. Flujo interno completo

1. Tráfico: La app o k6 envía solicitudes al NGINX del sistema simulado.

2. Balanceo: NGINX distribuye solicitudes entre App Backend #1 y #2.

3. Comportamiento: Los servicios responden normal, lento, con errores o dejan de responder.

4. Monitoreo: Monitoring Backend ejecuta health checks y recopila métricas.

5. Ventana de métricas: Se agregan latencia, error rate, timeouts, volumen y disponibilidad.

6. Análisis: Motor de reglas y AI Service evalúan el comportamiento.

7. Estado: Detection/State Engine determina Operativo, Degradado o Indisponible.

8. Incidente: Si corresponde, se crea/actualiza un incidente sin duplicarlo.

9. Persistencia: Supabase guarda evidencia relevante e historial; Redis mantiene estado/eventos.

10. Comunicación: Se generan alertas al dashboard y notificaciones a usuarios.

11. Recuperación: Cuando el servicio se estabiliza, el incidente pasa a Resuelto y se comunica el restablecimiento.

11. Escenarios de demostración

Escenario	Acción	Qué debe ocurrir	Qué demuestra
Normal	Carga baja/moderada	Estado Operativo, sin incidente.	Línea base.
Alta carga	k6 aumenta VUs/RPS	Suben latencia/errores; puede aparecer degradación.	Observabilidad y capacidad.
Degradación	Introducir latencia/errores/timeouts	IA/reglas detectan anomalía y se genera alerta preventiva.	Detección temprana.
Caída de una instancia	Detener App Backend #1	NGINX usa #2; app sigue respondiendo.	Failover/balanceo.
Caída total	Detener ambas instancias o servicio	Incidente crítico + comunicación de indisponibilidad.	Gestión de interrupción.
Recuperación	Levantar servicio y confirmar estabilidad	Incidente Resuelto + aviso de restablecimiento.	Cierre del ciclo.

12. Dashboard propio vs Grafana

	Dashboard del proyecto	Grafana
Usuario objetivo	Operador/administrador	Equipo técnico/TI
Contenido	Servicios, incidentes, severidad, IA, historial, MTTD/MTTR	CPU, RAM, requests, latencia, contenedores, Redis
Fuente	Monitoring Backend	Prometheus
¿Es el producto?	Sí, es la interfaz operativa del producto	Es una herramienta de observabilidad técnica

13. Reglas técnicas que debemos respetar

- La app móvil y el dashboard no se conectan directamente a PostgreSQL/Supabase; pasan por sus backends.

- No subir DATABASE_URL, claves de Supabase, JWT secrets ni tokens a Git. Usar .env y .env.example.
- Las migraciones de base de datos se versionan en Git (Prisma), no se coordinan con cambios manuales improvisados.
- No almacenar en PostgreSQL cada petición de k6. Usar agregación de métricas y herramientas apropiadas.
- La IA apoya la detección; la decisión final la toma el motor de estado con reglas + métricas + IA.
- El sistema supervisado debe poder fallar sin tumbar la plataforma de monitoreo.
- NGINX balancea tráfico; no es responsabilidad de la IA decidir el reparto normal.
- Cloudflare Tunnel es para desarrollo/demostración, no debe considerarse infraestructura de producción.

14. Cómo acelerar el desarrollo con Codex sin perder orden

La separación por repositorios permite dar a Codex un contexto pequeño y específico. En vez de pedir “construye todo el proyecto”, se trabaja por módulo y con criterios de aceptación.

Repositorio	Ejemplo de tarea para Codex
interbank-app-backend	Crear health endpoint y modo de simulación de latencia/error.
monitoring-backend	Implementar módulo incidents que evita incidentes activos duplicados por servicio.
monitoring-ai	Crear /predict que devuelva anomalyScore y riskLevel.
monitoring-dashboard	Construir vista de incidentes consumiendo la API del Monitoring Backend.
monitoring-infrastructure	Levantar NGINX + dos instancias del app backend + Redis + Prometheus en Compose.

15. Cómo aporta cada especialidad

Perfil	Aporte principal
Ingeniería de Software	Arquitectura, APIs, frontend/mobile, calidad, pruebas, CI/CD y mantenibilidad.
Sistemas / Computacional	Modelado, algoritmos, IA, anomalías, evaluación, falsos positivos y datos.
Tecnologías de Información	Docker, NGINX, redes, observabilidad, disponibilidad, Redis, Prometheus/Grafana.
Ingeniería Empresarial	Proceso de incidentes, KPI, valor para usuario, MTTD/MTTR, comunicación y mejora del proceso reactivo.

16. Preguntas frecuentes del equipo

¿El proyecto es la app móvil?

No. La app es una interfaz/prototipo para demostrar cómo recibe información el usuario.

¿El proyecto es el dashboard?

No. El dashboard es la interfaz del producto. El núcleo es la plataforma de monitoreo.

¿Hay dos backends?

Hay dos tipos: backend de la app simulada y Monitoring Backend. Además, el backend simulado puede ejecutarse en varias instancias.

¿Backend #1 y #2 son dos repositorios?

No. Son dos contenedores de la misma imagen, construida desde un solo repositorio y un solo Dockerfile.

¿Dónde está la IA?

En un AI Service separado (FastAPI/Python) integrado con el Monitoring Backend.

¿La IA controla NGINX?

No. NGINX hace balanceo/failover. La IA analiza anomalías.

¿Dónde se guardan los incidentes?

En PostgreSQL de Supabase, a través del Monitoring Backend.

¿Supabase ejecuta el backend?

No. Supabase se usa solo como PostgreSQL remoto compartido.

¿Qué hace Docker?

Construye/ejecuta los servicios y los conecta en un entorno reproducible.

¿Qué hace Cloudflare Tunnel?

Expone temporalmente una API/NGINX local a Internet mientras la PC anfitriona está encendida.

¿Por qué Redis si ya hay PostgreSQL?

Redis maneja estado rápido, cache, eventos y colas; PostgreSQL conserva información persistente.

¿La simulación llena la base de datos?

No si se agregan las métricas y se evita insertar cada request individual.

17. Orden de implementación recomendado

Fase 0 - Repositorios e infraestructura base - Crear los 6 repositorios, convenciones, .env.example, Dockerfiles mínimos y Compose.

Fase 1 - Sistema supervisado mínimo - App backend con Login/Saldo/Transferencia simulados + /health; NGINX con dos instancias.

Fase 2 - Monitoring Backend - Registrar servicios, ejecutar health checks, estado y métricas básicas.

Fase 3 - Incidentes - Crear, actualizar, resolver incidentes y persistir historial en Supabase.

Fase 4 - Dashboard - Visualizar estado, incidentes y detalle.

Fase 5 - Redis/notificaciones - Eventos, colas, deduplicación, push/in-app.

Fase 6 - IA - Dataset de simulación, Isolation Forest, /predict e integración con Detection Engine.

Fase 7 - Observabilidad y carga - Prometheus, Grafana, k6, escenarios de failover y degradación.

Fase 8 - Mobile y demo integrada - Mensajes al usuario, Cloudflare Tunnel y guion de demostración.

18. Decisiones de arquitectura congeladas

- Producto principal: plataforma de monitoreo y gestión proactiva de interrupciones.
- Dos dominios: sistema supervisado de prueba y plataforma de monitoreo.
- Backend simulado: NestJS; varias instancias del mismo repositorio.
- Monitoring Backend: NestJS modular.
- IA: microservicio interno FastAPI/Python/Scikit-learn.
- Base persistente: Supabase PostgreSQL remoto.
- ORM/migraciones: Prisma en los backends que correspondan.
- Redis: cache, colas y eventos temporales.
- NGINX: load balancer del sistema supervisado.
- Docker Compose: orquestación local en monitoring-infrastructure.
- Prometheus + Grafana: observabilidad técnica.
- k6: generación de carga y escenarios de prueba.
- Cloudflare Tunnel: acceso remoto de desarrollo/demostración.
- 6 repositorios independientes coordinados por infraestructura.

Idea final: el sistema supervisado puede fallar; la plataforma de monitoreo debe detectar, explicar, registrar, comunicar y verificar la recuperación.

19. Actualización del componente de IA

Estado a 09/10/2026: base funcional de monitoreo, no detector de fraude ni modelo predictivo entrenado.

Esta actualización precisa las secciones 5, 10 y 17 del informe original. El objetivo del componente de IA es detectar anomalías en la salud técnica de los servicios Login, Balance y Transferencias. No inspecciona cada operación bancaria, no decide si una transferencia es legítima y no detecta fraude. En este contexto, 'confiabilidad' significa disponibilidad y rendimiento del servicio, no confiabilidad de un cliente o una transacción.

19.1 Flujo automático implementado

Paso	Responsable	Comportamiento actual
1	Monitoring Backend	Cada 10 s (configurable) consulta los health endpoints a través de NGINX; evita rondas solapadas.
2	Ventana de métricas	Agrupa comprobaciones de los últimos 60 s. Requiere al menos 3 observaciones. Calcula p95, proporción de errores/timeouts y disponibilidad.
3	Prometheus	Aporta tasa de solicitudes, proporción de HTTP 5xx y p95 de tráfico real cuando hay series disponibles; si falla, continúan los health checks.
4	Monitoring AI	Recibe una ventana agregada, no transacciones individuales; devuelve anomalyScore, nivel de riesgo, modo y razones.
5	Detection + State	Combina reglas, observaciones y señal de IA. El backend decide OPERATIONAL, DEGRADED, UNAVAILABLE o RECOVERING.
6	Incidentes	PostgreSQL conserva un incidente activo por servicio y alertas de apertura, escalamiento y recuperación.

Principio de control: la IA aporta evidencia, pero no declara por sí sola una caída, no modifica NGINX y no abre incidentes directamente. Si Python o Prometheus no responden, el monitoreo por reglas continúa.

19.2 Dos capacidades, un servicio Python

Análisis automático: evaluación de ventanas invocada por Monitoring Backend tras reunir datos suficientes. Asistente de consulta: explicación en lenguaje natural a petición de un empleado autorizado; recibe evidencia seleccionada por Monitoring Backend y no participa en el ciclo de decisión automática.

20. Contratos y estado real del modelo

Endpoint de Monitoring AI	Función y alcance
GET /health	Confirma que FastAPI está activo; no indica que todo el ecosistema esté sano.
POST /predict	Contrato previo de compatibilidad para una muestra de métricas.
POST /v1/anomalies/analyze	Analiza una ventana agregada. Con menos de 3 observaciones devuelve INSUFFICIENT_DATA.
POST /v1/assistant/answer	Responde a una pregunta con la evidencia recibida. El proveedor none usa un resumen determinista.

20.1 Baseline actual frente a modelo futuro

El modo activo por defecto es baseline: una fórmula heurística pondera latencia, tasa de errores y tasa de timeouts. No ha aprendido de caídas históricas. Los umbrales de la API clasifican LOW, MEDIUM y HIGH; anomalyScore no es la probabilidad de que el servicio se caiga.

Ejemplo reproducido en Swagger: con p95 = 2400 ms, errorRate = 0,8 y timeoutRate = 0,6, la puntuación baseline es 0,35 + 0,28 + 0,18 = 0,81 (HIGH). Las razones indican errores, timeouts, latencia alta y health checks fallidos. Ese resultado corresponde a datos de prueba enviados manualmente, no a una transacción real ni a un fraude.

El código contempla Isolation Forest como opción posterior. Para activarlo se necesitan ventanas representativas, un artefacto entrenado y una evaluación temporal separada (precision, recall, falsos positivos y falsos negativos). No se debe presentar el baseline como un modelo entrenado ni prometer predicción anticipada de 15 minutos.

20.2 Asistente conversacional

Con ASSISTANT_PROVIDER=none, el endpoint ofrece un resumen determinista del estado e incidentes proporcionados. OpenAI o Gemini pueden configurarse opcionalmente con claves del lado servidor; si el proveedor falla, se usa el resumen de respaldo. Monitoring Backend expone POST /assistant/ask con autenticación, permiso analytics.read y límite de solicitudes. El asistente debe usar solo evidencia operativa permitida, no datos personales bancarios.

Prueba aislada: al ejecutar solo FastAPI, Swagger acepta métricas y evidencia manuales. La conexión continua con el sistema supervisado ocurre únicamente cuando se levanta Monitoring Backend y la infraestructura.

21. Validación, límites y siguiente etapa

21.1 Evidencia de implementación a la fecha

Comprobación	Resultado registrado
Pruebas automatizadas	5 de Monitoring AI, 16 de Monitoring Backend y 79 del App Backend aprobadas.
Calidad y construcción	Build de ambos backends, TypeScript, ESLint, Ruff y formato aprobados.
Persistencia	Migración de incidentes/alertas validada de manera aislada; aún falta desplegarla y probarla contra PostgreSQL real.
Velocidad local	10 000 inferencias baseline en proceso: aproximadamente 0,03 s en la medición realizada. No mide red, contenedores ni carga concurrente.
Resiliencia	Timeout de 800 ms para análisis de IA; reglas operativas continúan cuando IA o Prometheus no están disponibles.

21.2 Lo que aún NO está demostrado

No se ha validado todavía el ciclo extremo a extremo con Docker, NGINX, Prometheus y PostgreSQL real en esta entrega. Tampoco existe un dataset histórico etiquetado ni un modelo entrenado validado para anticipar caídas; los proveedores conversacionales externos no se probaron sin claves. El sistema no contiene un detector de fraude.

21.3 Criterios para avanzar

Etapa	Criterio de salida
Integración	Desplegar migraciones; observar estados e incidentes ante NORMAL, degradación, DOWN y recuperación en ambas instancias.
Datos	Recolectar ventanas normales y de fallo sin guardar cada request; etiquetar incidentes y separar evaluación posterior en el tiempo.
Modelo	Entrenar y comparar contra baseline; aceptar solo si mejora detección sin una tasa de falsas alarmas inaceptable.
Rendimiento	Medir latencia p95 extremo a extremo y comportamiento bajo carga real. La microprueba local no es criterio suficiente.
Operación	Probar failover, caída de IA/Prometheus y permisos del asistente antes de una demostración de producción.

Nota de vigencia del repositorio: la implementación actual reside en directorios de un mismo repositorio (interbank-monitoring-platform). La rama feature/monitoring-ai nace de develop; main permaneció intacta. Las menciones a seis repositorios independientes en el diseño inicial deben leerse como separación lógica de componentes.